Social Network Analysis (CS342) | Assignment 4

Analysis of Ego Facebook Network Dataset

Student Name: *Naishadh Rana*

Roll No: U23CS014

```
import pathlib as pl
import gzip
import tarfile
import requests
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
from collections import Counter

plt.style.use("seaborn-v0_8-muted")
print("Imported")
```

```
Imported
```

Data sources

Using the Facebook dataset from SNAP:

- Combined graph: https://snap.stanford.edu/data/facebook_combined.txt.gz
- Ego networks: https://snap.stanford.edu/data/facebook.tar.gz

```
# Paths and filenames
data_root = pl.Path("data/facebook")
data_root.mkdir(parents=True, exist_ok=True)

combined_url = "https://snap.stanford.edu/data/facebook_combined.txt.gz"
ego_url = "https://snap.stanford.edu/data/facebook.tar.gz"

combined_path = data_root / "facebook_combined.txt.gz"
ego_tar_path = data_root / "facebook.tar.gz"
ego_extract_dir = data_root / "ego_networks"
ego_extract_dir.mkdir(parents=True, exist_ok=True)

print(f"Data root: {data_root.resolve()}")
print(f"Combined graph file: {combined_path.name}")
print(f"Ego archive file: {ego_tar_path.name}")
```

```
Data root: /Users/naish/Desktop/College/Sem6/SNA/Solutions/data/facebook
Combined graph file: facebook_combined.txt.gz
Ego archive file: facebook.tar.gz
```

```
# helper to download files, skips if already downloaded
def download(url, dest):
    if dest.exists():
        print(f"Already present: {dest.name}")
        return
    print(f"Downloading {dest.name}...")
    r = requests.get(url, stream=True, timeout=60)
    r.raise_for_status()
    with open(dest, "wb") as f:
        for chunk in r.iter_content(chunk_size=8192):
            if chunk:
                f.write(chunk)
    print(f"Done! ({dest.stat().st_size/1e6:.1f} MB)")
```

Part I — Combined Facebook graph (SNAP)

Analyze and display following network parameters using NetworkX library:

- Number of Nodes

- Number of Edges Degree distribution chart
- Average degree
- Diameter
- Density
- Average path length
- Global clustering coefficient
- Local clustering coefficient (take one node as example node, see its connections and verify local clustering coefficient obtained with formula)

```
download(combined_url, combined_path)
```

```
Already present: facebook_combined.txt.gz
```

```
with gzip.open(combined_path, "rt") as f:
    G = nx.read_edgelist(f, nodetype=int, create_using=nx.Graph()) # loading graph

print(f"Nodes: {G.number_of_nodes():,}")
print(f"Edges: {G.number_of_edges():,}")

components = list(nx.connected_components(G))
components_sizes = sorted([len(c) for c in components], reverse=True)
print(f"Connected components: {len(components)}; largest size: {components_sizes[0]:,}")
```

```
Nodes: 4,039
Edges: 88,234
Connected components: 1; largest size: 4,039
```

The graph is fully connected (1 component).

- Each node = user
- Each edge = friendship.

```
n = G.number_of_nodes()
m = G.number_of_edges()
avg_degree = (2 * m) / n
density = nx.density(G)
global_clustering = nx.transitivity(G)
avg_clustering = nx.average_clustering(G)

largest_cc = max(nx.connected_components(G), key=len)
Gcc = G.subgraph(largest_cc).copy()
diameter = nx.diameter(Gcc)
avg_path_length = nx.average_shortest_path_length(Gcc)

metrics = {
    "nodes": n,
    "edges": m,
    "avg_degree": avg_degree,
    "density": density,
    "global_clustering": global_clustering,
    "avg_clustering": avg_clustering,
    "diameter (LCC)": diameter,
    "avg_path_length (LCC)": avg_path_length,
}

for k, v in metrics.items():
    if isinstance(v, float):
        print(f"{k}: {v:.4f}")
    else:
        print(f"{k}: {v}")
```

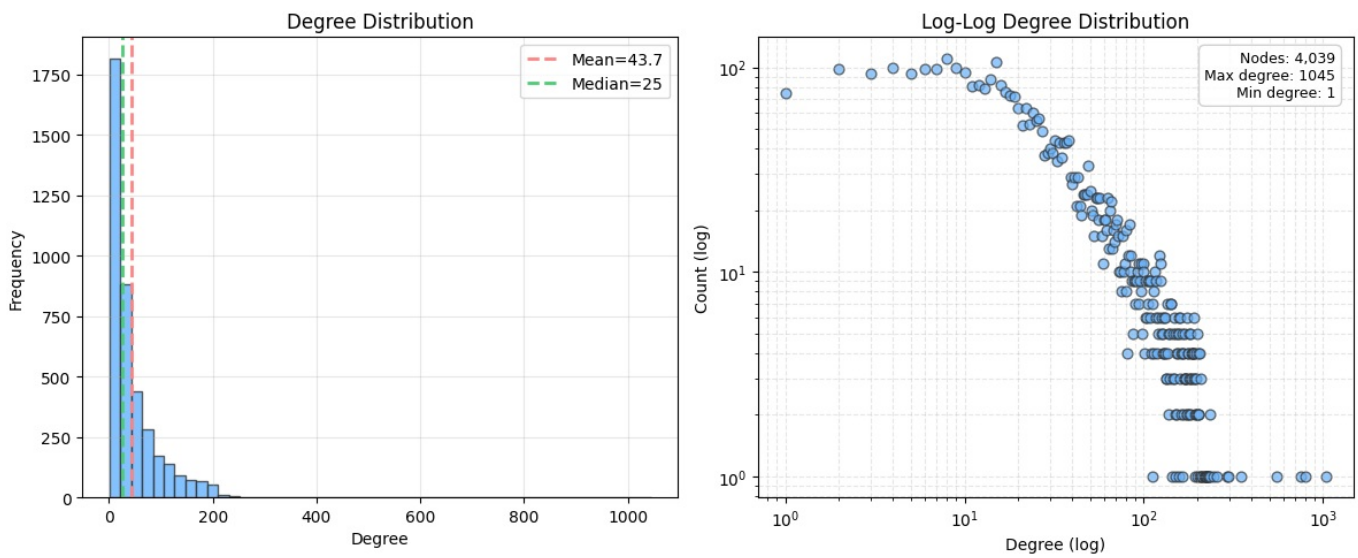
```
nodes: 4039
edges: 88234
avg_degree: 43.6910
density: 0.0108
global_clustering: 0.5192
avg_clustering: 0.6055
diameter (LCC): 8
avg_path_length (LCC): 3.6925
```

Formulas

Metric	Formula
Average degree	$\bar{k} = \dfrac{2m}{n}$
Density	$D = \dfrac{2m}{n(n-1)}$
Global clustering (transitivity)	$C = \dfrac{3 \times \text{triangles}}{\text{connected triples}}$
Average clustering	$\bar{C} = \dfrac{1}{n}\sum_{v} C_v$
Diameter	Longest shortest path in the graph
Average path length	$\ell = \dfrac{1}{n(n-1)} \sum_{u \neq v} d(u,v)$

Since the graph is connected, diameter and avg path length work directly. Else we'd use Largest Connected Component (LCC) to avoind inf distances.

```
# degree distribution plot
degs = [d for _, d in G.degree()]
deg_counts = Counter(degs)
xs = sorted(deg_counts.keys())
ys = [deg_counts[x] for x in xs]

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# histogram (linear)
ax1 = axes[0]
ax1.hist(degs, bins=50, color="#66b3ff", edgecolor="#333", alpha=0.8)
ax1.set_xlabel("Degree")
ax1.set_ylabel("Frequency")
ax1.set_title("Degree Distribution")
ax1.axvline(np.mean(degs), color="#FF8484", linestyle="--", lw=2, label=f"Mean={np.mean(degs):.1f}")
ax1.axvline(np.median(degs), color="#4ecd76", linestyle="--", lw=2, label=f"Median={np.median(degs):.0f}")
ax1.legend()
ax1.grid(alpha=0.3)

# log-log plot to check for power law
ax2 = axes[1]
ax2.scatter(xs, ys, s=40, c="#66b3ff", edgecolors="#333", alpha=0.7)
ax2.set_xlabel("Degree (log)")
ax2.set_ylabel("Count (log)")
ax2.set_title("Log-Log Degree Distribution")
ax2.set_xscale("log")
ax2.set_yscale("log")
ax2.grid(True, which="both", alpha=0.3, linestyle="--")

stats_text = f"Nodes: {len(degs):,}\nMax degree: {max(degs)}\nMin degree: {min(degs)}"
ax2.annotate(stats_text, xy=(0.97, 0.97), xycoords="axes fraction",
             ha="right", va="top", fontsize=9,
             bbox=dict(boxstyle="round,pad=0.4", fc="white", ec="#ccc", alpha=0.9))

plt.tight_layout()
plt.show()
```



The log-log plot shows a **heavy-tailed** distribution typical of real social networks. Most nodes have low degree (Few/close firends) while a few "hubs" (big social media presences) have very high degree. This is characteristic of **scale-free networks** where $P(k) \propto k^{-\gamma}$.

```
# verifying local clustering coefficient manually
# picking the node with highest degree as example
node = max(G.degree, key=lambda x: x[1])[0]
neighbors = list(G.neighbors(node))
k = len(neighbors)

# count edges between neighbors
edges_between = G.subgraph(neighbors).number_of_edges()
max_possible = k * (k-1) / 2

# calculate manually
my_cc = edges_between / max_possible if max_possible > 0 else 0
nx_cc = nx.clustering(G, node)

print(f"Node: {node}")
print(f"Degree: {k}")

print(f"NetworkX clustering: {nx_cc:.4f}")
print(f"Manual clustering: {my_cc:.4f}")

# print(f"Edges between neighbors: {edges_between}")
# print(f"Max possible edges: {int(max_possible)}")
```

```
Node: 107
Degree: 1045
NetworkX clustering: 0.0490
Manual clustering: 0.0490
```

Local clustering coefficient

Formula: $C_v = \dfrac{e}{k(k-1)/2}$

where:

- k is the degree of node v
- e is the number of edges among neighbors of v.

Basically asking: what fraction of my friends are also friends with each other? Verified above that my manual calc matches networkx.

Part I findings:

- The combined Facebook graph has **4,039 nodes** and **88,234 edges**.
- It forms a **single connected component**.
- Average degree ~ 43.7 — typical of dense social networks.
- High clustering coefficient (~0.6) indicates many closed triangles (friends of friends are also friends).
- Small diameter (8) and short average path length (~3.7). **small-world** property.

Part II — Ego-centric analysis (SNAP ego networks)

- Identify ego nodes
- For each ego node, extract its ego network and analyze its properties such as clustering coefficient, number of triangles, average path length and diameter
- Visualize Ego networks using NetworkX and Matplotlib

```
# get ego networks
download(ego_url, ego_tar_path)
```

```
Already present: facebook.tar.gz
```

```
# extract the tar.gz
if ego_tar_path.exists():
    with tarfile.open(ego_tar_path, "r:gz") as tar:
        tar.extractall(path=ego_extract_dir)
    print(f"Extracted to {ego_extract_dir}")
else:
    print("File not found, run download cell first")
```

```
Extracted to data/facebook/ego_networks
```

```
/var/folders/cg/qnsh7wx16ms111sfjgg47xxw0000gn/T/ipykernel_32291/1322946616.py:4: DeprecationWarning: Python 3.1
4 will, by default, filter extracted tar archives and reject files or modify their metadata. Use the filter argu
ment to control this behavior.
  tar.extractall(path=ego_extract_dir)
```

```
# find all .edges files (each one is an ego network)
ego_files = sorted(ego_extract_dir.rglob("*.edges"))
ego_ids = [int(f.stem) for f in ego_files]
print(f"Found {len(ego_files)} ego networks")
print(f"Ego IDs: {ego_ids}")
```

```
Found 10 ego networks
Ego IDs: [0, 107, 1684, 1912, 3437, 348, 3980, 414, 686, 698]
```

What's an ego network?

- **In general**: An **ego network** consists of a central node (**ego**) and all nodes directly connected to it (**alters**), plus any edges among the alters.
- **For our case**: Ego Network = one person (the **ego**) + all their friends (**alters**) + friendships between those friends.
- Ego-centric analysis helps understand local structure around *specific users*.

```
# function to analyze one ego network
def analyze_ego(filepath):
    ego_id = int(filepath.stem)
    g = nx.read_edgelist(filepath, nodetype=int)
    # add the ego node if not there
    if ego_id not in g:
        g.add_node(ego_id)

    triangles = sum(nx.triangles(g).values()) // 3
    cc = nx.average_clustering(g)
    # for path metrics, use largest component if disconnected
    if nx.is_connected(g):
        apl = nx.average_shortest_path_length(g)
        diam = nx.diameter(g)
    else:
        big = g.subgraph(max(nx.connected_components(g), key=len)).copy()
        apl = nx.average_shortest_path_length(big)
        diam = nx.diameter(big)

    return {
        "ego": ego_id,
        "nodes": g.number_of_nodes(),
        "edges": g.number_of_edges(),
        "triangles": triangles,
        "clustering": cc,
        "avg_path": apl,
        "diameter": diam,
    }
```

```
# analyze all ego networks
import pandas as pd
```

```
results = [analyze_ego(f) for f in ego_files]
df = pd.DataFrame(results).set_index("ego")
df = df.round(4)
df
```

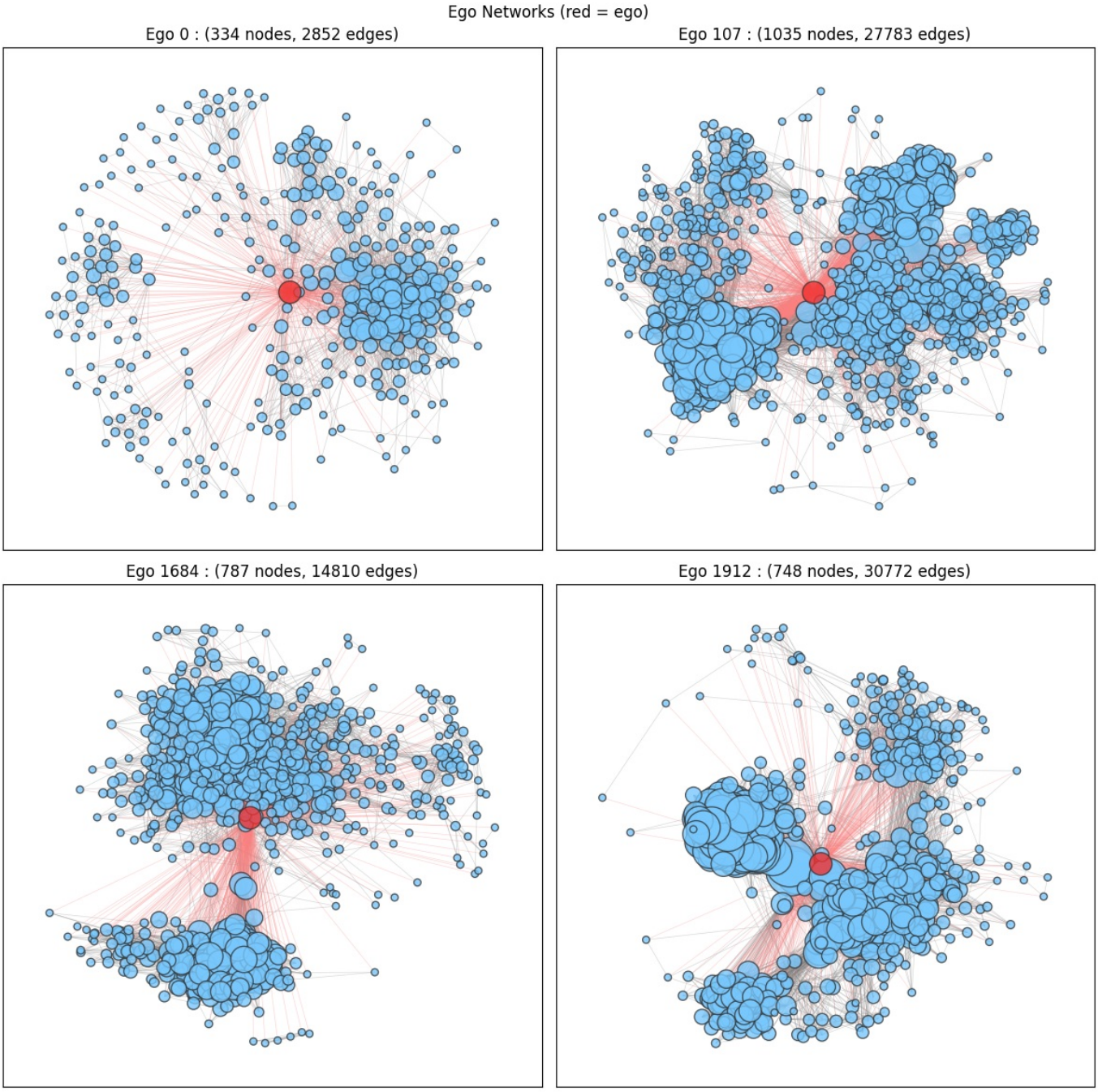
ego	nodes	edges	triangles	clustering	avg_path	diameter
0	334	2519	10740	0.5067	3.7527	11
107	1035	26749	420329	0.5259	2.9517	9
1684	787	14024	121532	0.4751	3.0423	10
1912	748	30025	916277	0.6346	2.5584	7
3437	535	4813	20849	0.5427	3.4474	9
348	225	3192	23503	0.5419	2.5235	9
3980	53	146	168	0.4530	2.5666	5
414	151	1693	10618	0.6659	2.6916	7
686	169	1656	7945	0.5306	2.4251	6
698	62	270	679	0.7214	1.9449	4

What these metrics mean

Metric	Meaning
Triangles	Number of closed triplets — indicates tight-knit groups
Avg clustering	How connected a node's neighbors are to each other
Avg path length	Typical separation/hops between nodes in the ego network
Diameter	Maximum shortest path, indicates network "spread"

High clustering + low diameter → dense, tight and cohesive ego network.

```
# visualization of first 4 ego networks
fig, axes = plt.subplots(2, 2, figsize=(12, 12))

for i, ax in enumerate(axes.flatten()[:min(4, len(ego_files))]):
    fpath = ego_files[i]
    ego_id = int(fpath.stem)
    g = nx.read_edgelist(fpath, nodetype=int)

    alters = list(g.nodes())
    g.add_node(ego_id)
    for a in alters:
        g.add_edge(ego_id, a)

    pos = nx.spring_layout(g, seed=42, k=2/np.sqrt(len(g)), iterations=50)
    pos[ego_id] = np.array([0, 0])

    colors = ["#F13232DD" if n == ego_id else "#79C9FFDD" for n in g.nodes()]
    sizes = [300 if n == ego_id else max(30, g.degree(n) * 5) for n in g.nodes()]

    ego_edges = [(ego_id, v) for v in g.neighbors(ego_id)]
    alter_edges = [(u, v) for u, v in g.edges() if u != ego_id and v != ego_id]

    nx.draw_networkx_nodes(g, pos, ax=ax, node_size=sizes, node_color=colors, alpha=0.8, edgecolors="#333")
    nx.draw_networkx_edges(g, pos, ax=ax, edgelist=alter_edges, edge_color="gray", alpha=0.3, width=0.5)
    nx.draw_networkx_edges(g, pos, ax=ax, edgelist=ego_edges, edge_color="#ff8181b6", alpha=0.3, width=0.5)

    ax.set_title(f"Ego {ego_id} : ({g.number_of_nodes()} nodes, {g.number_of_edges()} edges)")

plt.suptitle("Ego Networks (red = ego)")
plt.tight_layout()
plt.show()
```



- **Red node** = the ego (central user)
- **Blue nodes** = alters (friends of the ego)
- Node size scales with degree within that ego network (meaning bigger nodes have more connections)

Part II findings

- 10 ego networks identified, varying in size from ~50 to ~1,000+ nodes.
- All show high clustering, indicating tight local communities.
- Visualizations reveal community structure within each ego's neighborhood.